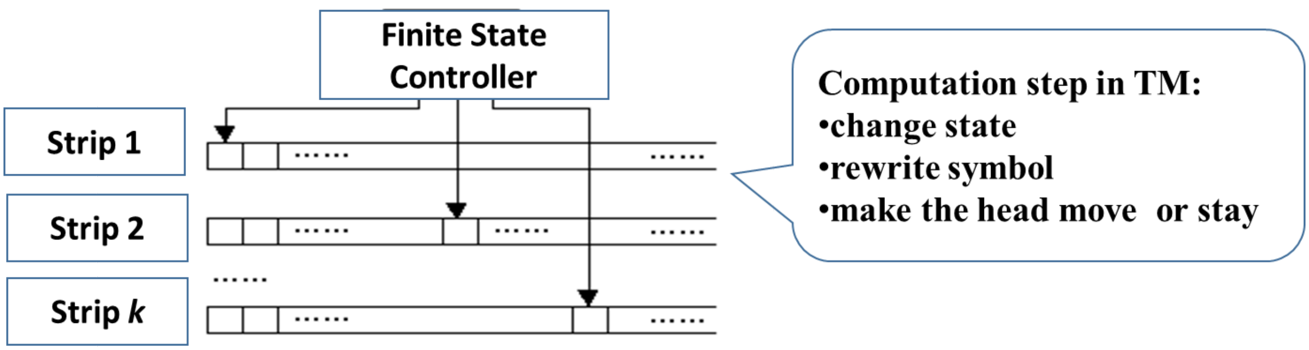
Lecture 8 NPC Problems

Turing Machine

A k-strip TM is a tuple $< Q, T, I, \delta, b, q_0, q_f >$.

- A set of finite states
- a set of finite symbols
- a set of input symbols and $I \subseteq T$
- a blank symbol
- an initial state
- a final state
- a transfer function: $Q \times T^k \to Q \times (T \times \{L, R, S\})^k$
  k is the number of strips



steps

- change state
- rewrite symbol
- make the head move or stay

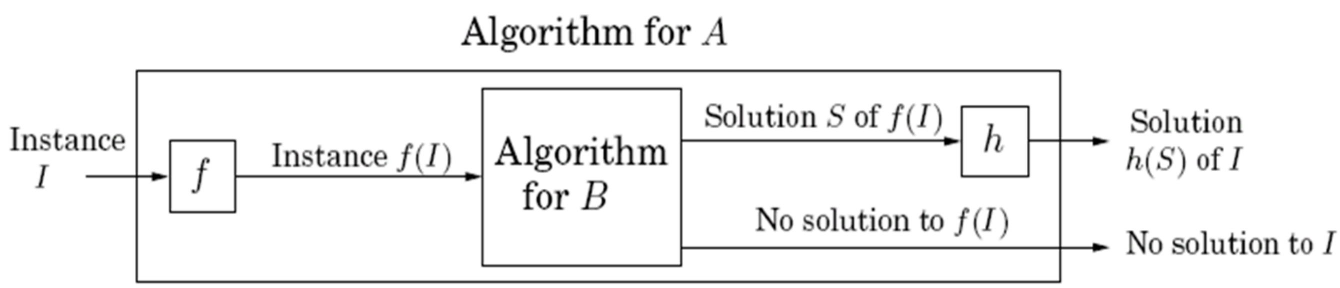
NP-complete problems

- P and NP:
  - NP is the class of all problems that can be solved in polynomial time on nondeterministic TM($Q \times T^k \to 2^Q \times T \times \{L, R, S\}$, i.e., one state can correspond to many following states)
  - P is the class of all problem that can be solved in polynomial time on deterministic TM.
  - NPC is the class of hardest problems in NP. (NPC $\subseteq$ NP)

Reduction

A reduction from A to B is $f + h$

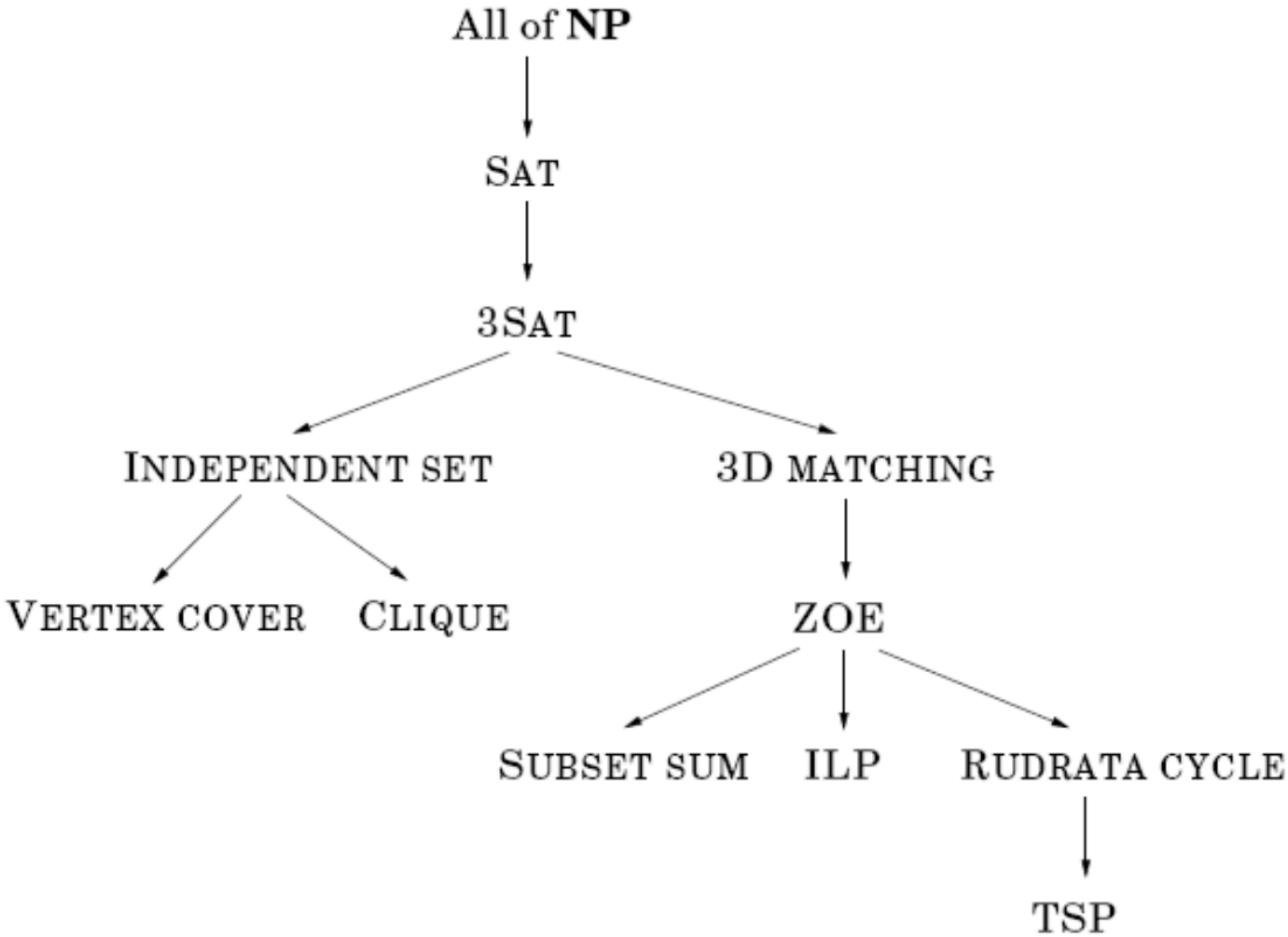
- both the two functions are polynomial-time algorithms
- f transforms any instance I of A into a instance $f(I)$ of B
- h maps any solution S of $f(I)$ back into a solution $h(S)$ of I.



If B have no solution, A will also have no solution. $A \leq_P B$ is that A can be polynomial-time reduction to B. B is harder than A. If we solve B, we can solve A.
We can use reduction

- Design algorithm(upper bound): A is hard but B has a efficient polynomial-time algorithm. If $A \leq_P B$, A must have a polynomial-time solution
- Prove NP-hard(lower bound): To prove A is hard, find a $B \leq_P A$ and B is a known NP-hard problem
- Prove NP-Complete: Prove A belongs to NP and a known NPC can be reduction to A.
  So, if one of NPC has a polynomial time algorithm, then every problem in NP can be solved in polynomial time, i.e., $P = NP$. It is hard to prove.

Reduction Tree



cook theorem: SAT is the first NPC problem.
SAT problem: give the Boolean formula that is conjunctive($a_1 \vee a_2 \vee \cdots \vee a_k$), judge whether their is a set value to make the formula be true.
3SAT problem: specific example of SAT problem. only three words or fewer in one clause.
Independent set: find a set of g pairwise non-adjacent vertices in graph